

【睿迄科技】全栈开发 3 天挑战

全栈挑战 · 后端架构与核心业务逻辑

在你开始之前

我们想看的不是你能不能「跑通」一个表单,而是——你能不能把一个看似简单的健康测评,从数据库到接口到鉴权,搭出一个站得住的工程骨架,并且自己证明它是对的。

API 设计的合理性、数据建模的稳健性、流程的闭环、你怎么保证质量(测试与边界覆盖)、以及你和 AI 的协作效率,是我们最关心的五件事。

补一句关于「保证质量」:在我们团队没有单独的测试岗,工程师自己用 AI 把测试与边界覆盖做扎实——所以这道题里,写测试不是加分项,是基本盘。

UI 不在考查范围内,能跑就行。

关于时间:我们把这道题从过去的 5 天压到了 3 天,但要求只增不减(还多了测试)。原因很简单——在 AI 时代,我们相信一个足够强的工程师 + AI,能在更短时间里交付过去要更久才能完成的东西。你怎么用 AI 在 3 天里把「骨架 + 测试」都做扎实,本身就是我们想看的。

一、参考竞品

请先体验一下这个产品,了解一下完整的测评流程和数据结构,再开始动手——你将要搭建的后端,就是支撑这种产品的核心基础设施:



BetterMe

Quiz

Funnel:<https://betterme-pilates.com/first-page-brand-palette?flow=2117>

我们不要求你 1:1 复刻它的功能,但希望你认真观察它的数据流——哪些步骤产生哪些数据、哪些数据需要持久化、订阅前后用户能拿到什么——并在你的设

计里做出你认为更合理的判断。

二、任务目标

设计并开发一个健康测评系统的核心后端架构。我们看重的是：

- API 设计规范:路径、方法、请求 / 响应结构是否专业
- 数据库建模稳定性:表结构是否可扩展、字段选择是否合理
- 数据持久化逻辑:分步保存、进度恢复、状态一致性
- 模拟订阅体系:鉴权、差异化返回、支付回调闭环

测试与质量保障:核心逻辑与关键流程有自动化测试覆盖,边界与异常路径被显式验证

三、核心流程(后端导向)

第一阶段 · 测评数据流与状态恢复(Persistence)

功能点:用户填写性别、目标、身体数据(年龄、身高、体重、目标体重)及运动频率。

技术要求:

- 分步保存接口:用户每完成一步,前端调用接口将增量数据同步至后端
- 进度恢复逻辑:用户中途关闭页面再次进入时,后端需返回该用户已填写的进度数据(可基于随机生成的 UserID 或简易 Session 识别)

第二阶段 · 服务端计算逻辑(Core Logic)

功能点:用户提交所有数据后,触发后端逻辑处理。

技术要求:

- 在服务器端实现一套简单的「健康评估算法」,计算出 BMI、建议摄入

量、目标预测日期

- 将计算结果持久化存储,并关联至用户记录

第三阶段 · 订阅鉴权与权限保护(Auth & Access)

功能点:结果页展示。

技术要求:

- 逻辑拦截:结果页接口需校验用户的 `subscription_status`(模拟状态)
- 差异化返回:
 - 非会员:API 仅返回部分脱敏数据(如隐藏具体的预测曲线数据),并提示需付费
 - 会员:API 返回完整数据
- 模拟支付回调:提供一个 `/pay` 接口,调用后修改数据库中的会员状态为有效

第四阶段 · 测试与质量保障

功能点:证明上面三个阶段是对的,而不只是「我本地点了一下没问题」。

技术要求:

- 用自动化测试覆盖核心逻辑与关键流程,至少包含:
 - 健康评估算法的单元测试(含边界:极端 / 缺失 / 非法的身高、体重、年龄,目标体重不合理等)
 - 分步保存 + 进度恢复的集成测试(中断后恢复、乱序 / 重复提交、并发更新)
 - 鉴权差异化返回的测试(非会员脱敏 vs 会员完整,确保非会员拿不到被保护字段)
 - `/pay` 回调后状态变更,以及结果页返回从「脱敏」变为「完整」的端到端验证

- 数据验证:接口要能挡住非法数值注入与越界输入,并对这些情况有测试覆盖
- 提供一键运行测试的方式(如 `npm test`);若能接入 CI(GitHub Actions 等)让测试自动跑起来并贴出通过状态,加分
- 在 README 里说明:你覆盖了哪些场景、为什么是这些、哪些暂时没覆盖以及原因

四、技术要求

- 前端:Next.js(App Router)或任何你擅长的框架。我们不评判动效细节和像素级还原,但前端要让真实用户愿意一路填到付费弹窗——基础视觉、文案、节奏、信任感不能糊弄,以吸引用户的角度考量
- 后端(Core):Node.js(Next.js API Routes / NestJS / Express 任选)+ TypeScript
- 数据库:Supabase / Prisma + PostgreSQL
- 测试:任选主流框架(Jest / Vitest / Playwright 等),覆盖第四阶段要求的核心逻辑与流程
- 部署:必须提供一个公网可达、可完整演示的线上链接(我们会直接在线上跑一遍 funnel + 模拟支付的全流程)

五、交付物

1. 完整可演示的线上链接(必填)
 - a. 公网可达的 URL,我们点进去能从头到尾走一遍 funnel
 - b. 含 `/pay` 接口的可重放调用方式(README 里给一段 cURL 或 Postman collection)
 - c. 提供一个已支付的测试 `sessionId`,让我们能直接对比付费前后的差异化返回

- 2. 代码仓库:GitHub 链接,附带 README 说明如何启动及 API 文档
- 3. 自动化测试(本次新增):测试代码随仓库提交,README 写明一键运行方式与覆盖范围;接入 CI 并贴出通过状态者加分
- 4. 数据库 Schema 图:展示用户表、数据记录表、订阅信息表之间的关系
- 5. AI 使用复盘:简述你如何利用 AI 进行数据库建模、生成 Mock 数据、编写复杂逻辑,以及生成测试用例 / 测试数据 / 边界场景;另请写一段「有没有一次 AI 给的方案或测试你判断不对、否决掉了?为什么?」

注意:前端要以吸引用户的角度考量。

六、评分标准

维度	我们在看什么
后端功底	API 路径设计是否合理?数据验证是否严谨(能否防止非法数值注入、边界值是否处理到位)?
DB 设计	表结构是否具备扩展性?字段选择是否专业?关系建模是否经得起推敲?
逻辑闭环	从录入 → 存储 → 权限校验 → 模拟支付,整个流程是否能完整跑通?刷新、中断、异常路径是否都考虑到了?
测试与质量(新增)	测试是否覆盖了核心逻辑与关键流程?用例是不是只测 happy path,还是真的覆盖了边界与异常?能否一键跑通 / 接入 CI?
AI 效率	你能否调教 AI 生成高质量的类型定义(Types)、数据库迁移脚本与测试用例?是不是只把 AI 当代码生成器,还是真的在和它协作?

关于时间与节奏

3 天,预期投入相当于一个紧凑的工作冲刺。建议节奏:

- 第 1 天:吃透 funnel 数据流 + 定 Schema + 搭起接口骨架(分步保存 / 进度恢复)
- 第 2 天:核心算法 + 鉴权差异化 + /pay 回调闭环,边写边补测试
- 第 3 天:补齐边界与异常测试、(可选)接 CI、部署上线、整理 README + AI 复盘

怎么用 AI 在 3 天里把「骨架 + 测试」都做扎实,本身就是这道题想评估的一部分。

交付方式

3 天内将交付物发送至 jin@arkon-tech.com、 bin@arkon-tech.com、alex@arkon-tech.com、rip@arkon-tech.com 四个邮箱。

文档命名格式:【姓名】_全栈挑战_YYYYMMDD

期待看到你的作品。后续的面试会以该结果为准,决定是否继续推进面试。表现卓越者【甚至可以直接进入终面录取】+ 更大的薪资回报,期待你的答案~